String Matching

An important feature of text editors is to be able to search for occurrences of a particular string in a text document. Efficient algorithms to solve this problem can greatly improve the performance of the application, particularly if the text document is large.

We first look at an efficient algorithm to solve this problem, due to Knuth, Morris and Pratt; commonly known as the **KMP String Matching algorithm**.

Setup

Define the **string matching problem** as follows:

Input: The text $T[1..n]$ and a pattern $P[1..m]$ $(m \leq n)$, which are strings over some alphabet Σ .

Output: Find all valid shifts with which P occurs in T, i.e. find all s such that $T[s+1..s+m] = P[1..m]$.

For example, if $T =$ "abcabaabcabac" and $P =$ "abaa", then $s = 3$ is a valid shift, in fact it is the only valid shift.

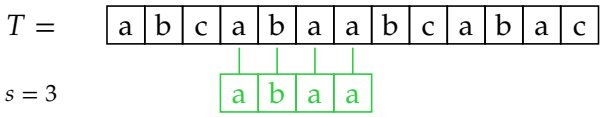


Figure 1: An example instance of the string matching problem

Naive Algorithm

As per the definition, we can try out all shift positions, and check if there is a match.

NAIVE-STRING-MATCH$(T[1..n], P[1..m])$

```
1  for i from 1 to n − m + 1
2      if T[i..i + m − 1] matches P[1..m]
3          output i − 1
```

This achieves our goal (is correct by definition). It takes $O((n-m+1)m)$ time, which is tight in the worst case. Some worked examples are shown below.

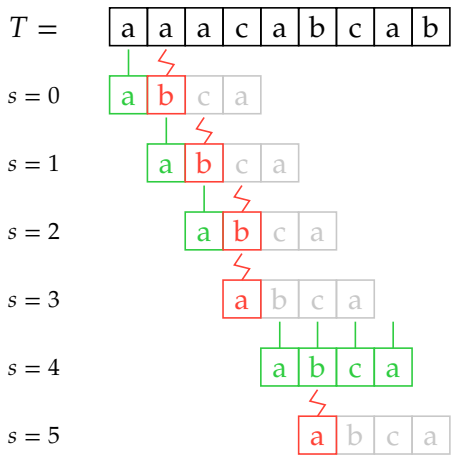


Figure 2: Naive string matching for $T =$"aaacabcab", $P =$"abca"

As shown in Figure 2, for each shift position s, we compare character by character the pattern and the text. If we find a mismatch, we stop and try the next shift. If we match the entire pattern then an occurrence has been found and we output that particular shift value.

How can we improve the running time of the algorithm?

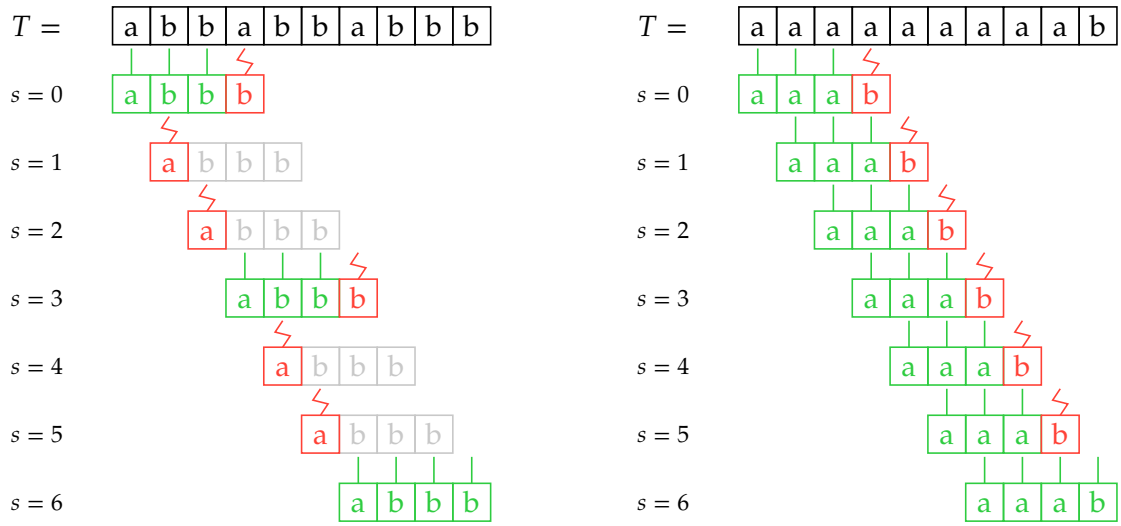


Figure 3: Naive string matching for (a) T ="abbabbabbb", P ="abbb" (b) T ="aaaaaaaaab", P ="aaab"

In Figure 3, we see two kinds of redundancies that we may try to optimise on in the Naive algorithm. In (a), shift $s = 0$ fails after matching $P[1..3] = \text{abb}$. So we know $T[1..3] = \text{abb}$. Shifts 1 and 2 would need $T[2]$ and $T[3]$ to equal $P[1] = \text{a}$, but both are b. These shifts cannot possibly be valid, yet the naive algorithm tries them anyway.

In (b), shift $s = 0$ fails after matching $P[1..3] = \text{aaa}$. The next shift cannot be ruled out, but we already know $T[2..3] = \text{aa}$, which matches $P[1..2]$. The naive algorithm still goes back and re-compares these characters.

The trick that we can use to reduce the number of comparisons comes from this fact: **when a mismatch occurs after matching q characters, the text we just read is known to be exactly $P[1..q]$.** So what to do next depends only on the pattern and on q, not on the text. This lets us

- skip shifts that the pattern shows cannot be valid, and
- never move backwards in T, so each text character is read only once.

Observations

Suppose we are checking shift s and the first q characters of the pattern match, but then we hit a mismatch at $T[i]$, where $i = s + q + 1$. So we know exactly two things:

- $T[s + 1..i - 1] = P[1..q]$, and
- $T[i] \neq P[q + 1]$.

Using only this, what is the smallest shift $s' > s$ that could still be valid?

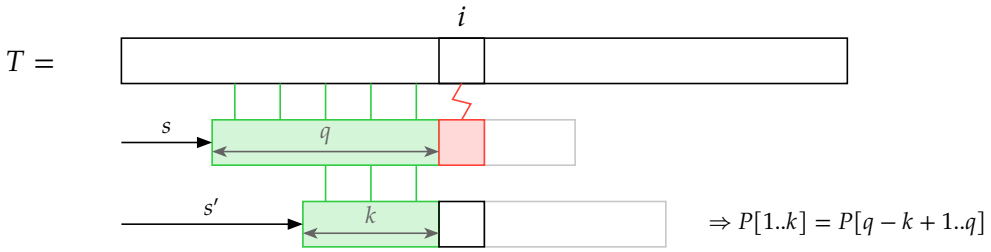


Figure 4: Key observation for the KMP algorithm

Figure 4 shows this situation. Suppose s' is a valid shift, and let k be the number of pattern characters that correspond to the part of the text before i.

For s' to be valid, these k characters must match the last k characters in the text before i. But, we know that these k characters are the ones which were earlier matched by the last part of $P[1..q]$.

Thus, by the diagram we can conclude this: For s' to be a valid shift, we require

$$P[1..k] = P[q - k + 1..q]$$

In other words, $P[1..k]$ is a proper prefix of $P[1..q]$, which is also a suffix of $P[1..q]$. This is a necessary condition, so we can skip those shifts which do not follow this.

We want the smallest such s' (as we don't want to skip over any valid shift); this means we want the largest such k. So, we need to compute **the longest proper prefix of $P[1..q]$ which is also its suffix**. Notice that this quantity does not involve T so we can precompute it for every q. Also, after shifting to s', we already know that the first k characters match, so we resume the comparison from $T[i]$ and never move back in the text.

LPS Table

Let the pattern be $P[1..m]$. Define the **longest prefix-suffix** $\text{LPS}[q]$ $(1 \leq q \leq m)$ as the **length of the longest proper prefix of $P[1..q]$ which is also a suffix of it.** $(0 \leq \text{LPS}[q] < q)$

For instance, consider the pattern $P =$"ababaca". We can compute the LPS table for this pattern as shown in Figure 5. E.g. $\text{LPS}[5] = 3$, since for the string $P[1..5] =$"ababa", its longest prefix suffix is "aba", whose length is 3. Note that $\text{LPS}[1] = 0$ always.

q	1	2	3	4	5	6	7
$P[q]$	a	b	a	b	a	c	a
$\text{LPS}[q]$	0	0	1	2	3	0	1

Figure 5: LPS table for $P =$"ababaca"

Suppose we have matched the first 5 characters of P, but we get a mismatch at $T[i]$ with $P[6]$. By Figure 4, the only shifts that can still be valid are those for which a prefix of $P[1..5] =$"ababa" is also a suffix of it. These are, from longest to shortest, "aba", "a" and ε (the empty string), and they correspond to the shifts in Figure 6. Each of them is consistent with the part of the text we have already matched.

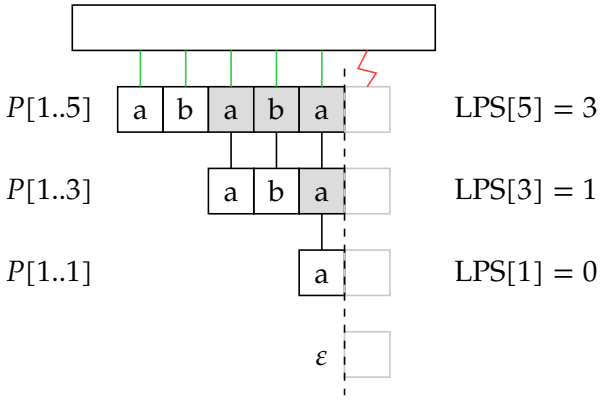


Figure 6: All proper prefix-suffixes of $P[1..5]$

All of this information is actually encoded in the LPS table itself!

Claim: The lengths of the proper prefix-suffixes of $P[1..q]$ are exactly $\text{LPS}[q], \text{LPS}[\text{LPS}[q]], \ldots, 0$.

Proof. Write $\text{LPS}^{(j)}[q]$ for LPS applied j times to q.

($\supseteq$) A prefix of a prefix is a prefix, and a suffix of a suffix is a suffix. So a proper prefix-suffix of $P[1..k]$ is also a proper prefix-suffix of $P[1..q]$ whenever $P[1..k]$ is one. Since $P[1..\text{LPS}[q]]$ is a proper prefix-suffix of $P[1..q]$, so every $P\left[1..\text{LPS}^{(j)}[q]\right]$ is also a proper prefix-suffix of $P[1..q]$, by induction on j.

($\subseteq$) Let $P[1..k]$ be a proper prefix-suffix of $P[1..q]$, and let $k' = \text{LPS}[q]$, so $k \leq k'$. If $k = k'$ we are done. Otherwise $k < k'$, and both $P[1..k]$ and $P[1..k']$ are suffixes of $P[1..q]$, so $P[1..k]$ is a suffix of $P[1..k']$. It is also a prefix of $P[1..k']$, so it is a proper prefix-suffix of $P[1..k']$. Again, an inductive argument shows k appears in the sequence. □

Therefore, after a mismatch at $T[i] \neq P[q+1]$, we will shift by $q - \text{LPS}[q]$ (why?) to get the next possible valid shift, and again try matching $T[i]$, and continue like so. Thus we can formulate the KMP algorithm.

The Matching Algorithm

The following is the matching algorithm for KMP, given we know how to compute the LPS table for P (which we will discuss shortly).

KMP-STRING-MATCH($T[1..n]$, $P[1..m]$)

```
1   LPS ← COMPUTE-LPS(P)
2   q ← 0
3   for i from 1 to n
4       while q > 0 and P[q + 1] ≠ T[i]
5           q ← LPS[q]
6       if P[q + 1] = T[i]
7           q ← q + 1
8       if q = m
9           output i − m
10          q ← LPS[q]
```

We scan T once from left to right and keep a number q, the length of the part of the pattern we are currently matching. Precisely, at the start of every iteration i, we have the **invariant**:

$$q \text{ is the length of the longest prefix of } P \text{ with } q < m \text{ that is a suffix of } T[1..i-1].$$

On each iteration:

- [Lines 4-5] If $T[i]$ is a mismatch, it can't extend the current matched $P[1..q]$. We try other shifts of the pattern which are consistent with what we have matched, by iterating over $\text{LPS}[q], \text{LPS}[\text{LPS}[q]], \ldots$ till we find a match or hit 0.
- [Lines 6-7] If we managed to match $T[i]$, then the match grows by one character.
- [Lines 8-10] If $q = m$, we have found a full match, so output the result. Then, we go to the next possible valid shift ($q \leftarrow \text{LPS}[q]$) and continue.

Figure 8 shows a full worked example of the algorithm.

Time Complexity

Let's discuss the time complexity of the matching algorithm (other than that of computing the LPS array). We will show that the running time is $\Theta(n)$. The only tricky part is showing that the **while** loop of lines 4-5 executes $O(n)$ times altogether, which we show by aggregate analysis.

We know that $\text{LPS}[q] < q$, so in each iteration of the **while** loop, q must strictly decrease. On each iteration of the outer **for** loop, q can increase at most once, on line 7. So, the value of q can never exceed n (since q starts off at 0 and there are n iterations of the for loop). Then it follows, that the total number of iterations of the **while** loop can not exceed n, since q never becomes negative (why?). Thus, the overall running time of the matching algorithm (except LPS computation) is $\Theta(n)$, i.e. linear time.

Correctness

We show that the invariant is maintained. Suppose it holds at the start of iteration i. At the end of this iteration we want q to be the length of the longest prefix of P which is a suffix of $T[1..i]$. Let this value be $k+1$, so $P[k+1] = T[i]$. Then, it should be the case that $P[1..k]$ is a suffix of $T[1..i-1]$. By the invariant (see Figure 4) and the claim, this means k should be one of $q, \text{LPS}[q], \text{LPS}[\text{LPS}[q]], \ldots, 0$. We visit each such possible value in the while loop in decreasing order and stop if we can match $T[i]$, thus we get the new longest match, hence the invariant holds. If we couldn't match $T[i]$ and hit 0, then the new q is 0. If we are able to match m characters, then we output the match and then restore the invariant by setting $q \leftarrow \text{LPS}[q]$.

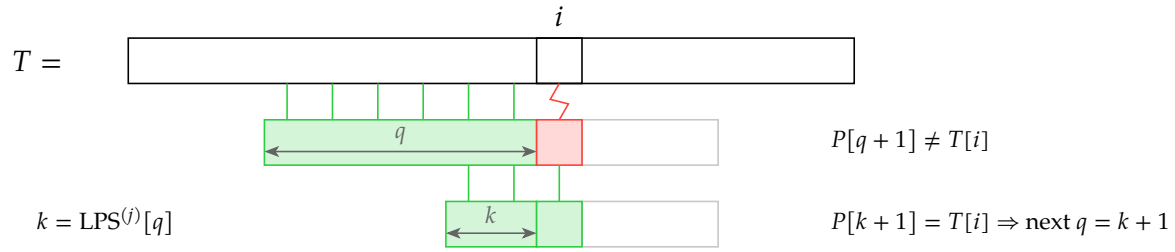


Figure 7: Maintaining the invariant. $P[1..q]$ is the longest prefix of P (with $q < m$) that is a suffix of $T[1..i-1]$. We want the longest such prefix which can be extended so that $T[i]$ is also matched.

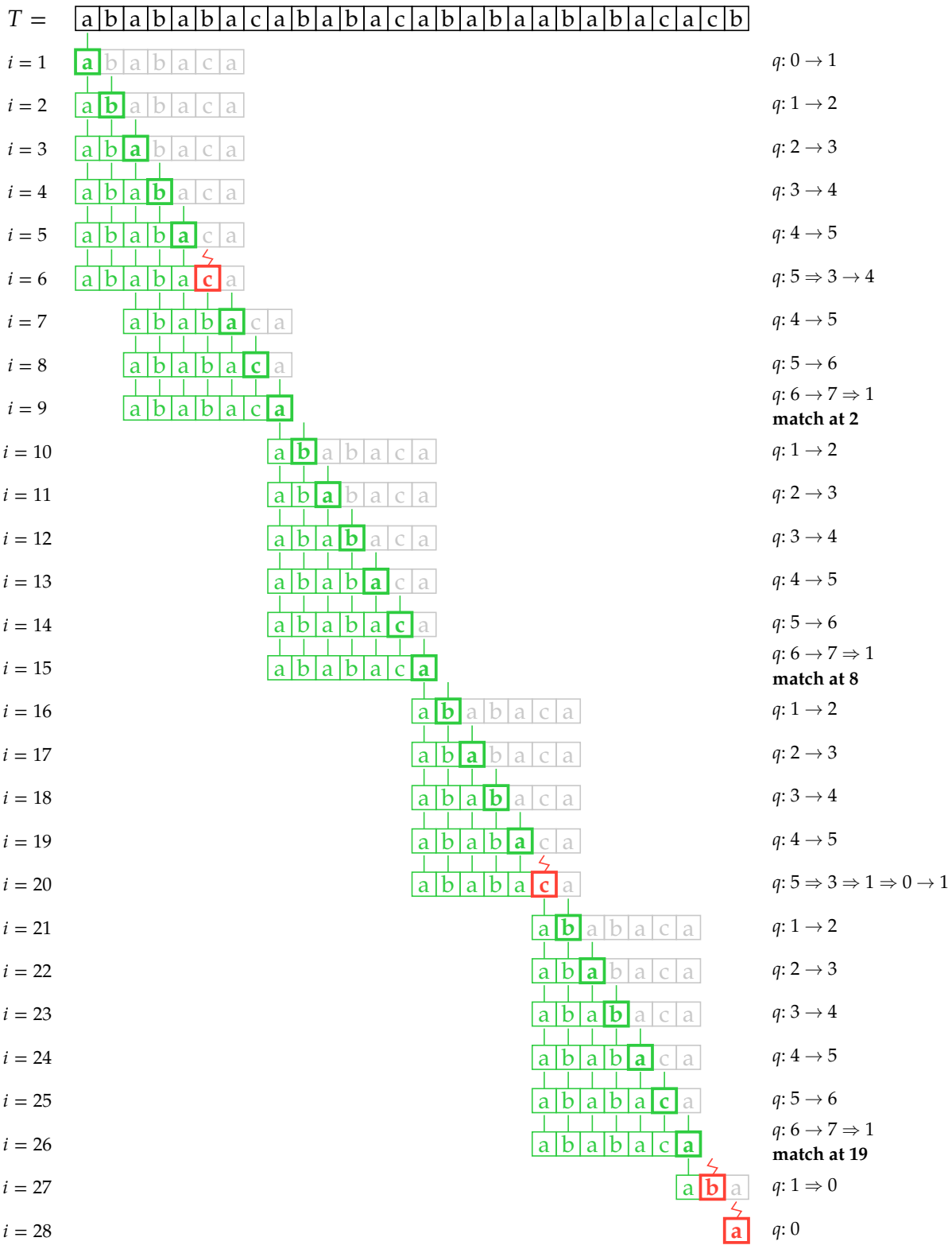


Figure 8: A worked out example of the KMP algorithm, for T ="abababacababacababaabababacacb" and P ="abababaca". Refer to the LPS table of P in Figure 5. On the right, changes to q show how the matching and shifts proceed. $\rightarrow$ represents an increment to q, and $\Rightarrow$ represents an update of q to LPS$[q]$.

Computing the LPS table

It turns out that computing the LPS values is very similar to the matching algorithm itself: instead of matching P with T, we match P with itself.

COMPUTE-LPS($P[1..m]$)

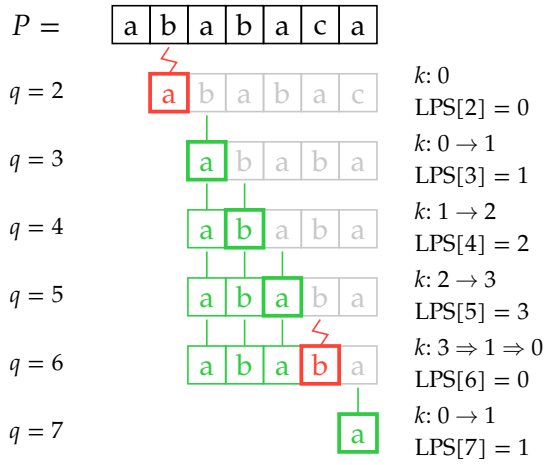
```
1  LPS[1] ← 0
2  k ← 0
3  for q from 2 to m
4     while k > 0 and P[k + 1] ≠ P[q]
5        k ← LPS[k]
6     if P[k + 1] = P[q]
7        k ← k + 1
8     LPS[q] ← k
9  return LPS
```

In each iteration of the for loop, q is the index in P for which LPS is to be computed, and as an invariant, at the start of each iteration q, we have

$$k = \text{LPS}[q-1]$$

In the iteration, we try to match $P[q]$ with $P[k+1]$ (so as to extend the previous match). If it doesn't work, we shrink k to the next possible value, and continue doing so until we are able to match $P[q]$.

See Figure 9 for an example of an LPS table computed for a pattern.



q	1	2	3	4	5	6	7
$P[q]$	a	b	a	b	a	c	a
LPS$[q]$	0	0	1	2	3	0	1

Figure 9: Computing the LPS table for P ="ababaca"

Correctness

Suppose that LPS$[j]$ for all $j < q$ has been computed correctly, and at the start of the qth iteration, $k = \text{LPS}[q-1]$. We wish to find LPS$[q]$, i.e. the length of the longest prefix-suffix of $P[1..q]$. Call this value $k'+1$. Then, $P[1..k']$ must be a suffix of $P[1..q-1]$. By the claim, this means $k' \in \{k, \text{LPS}[k], \text{LPS}[\text{LPS}[k]], \ldots, 0\}$. Thus, we iterate over each of these possibilities in decreasing order of length and stop if we are able to match $P[q]$. At this point, $k'+1$ is the length of the longest prefix-suffix of $P[1..q]$, so we store it. If we hit 0, then no proper prefix-suffix of $P[1..q]$ exists, so we store 0.

Time Complexity

By a similar aggregate analysis, we can show that this procedure runs in $\Theta(m)$ time.

In total, we get an overall time complexity of the KMP algorithm to be $\Theta(m + n)$, where $\Theta(m)$ is preprocessing time and $\Theta(n)$ is the actual time for matching a string.

Summary

We have discussed about the String Matching problem, first discussing the naive algorithm which runs in $O((n - m + 1)m)$ time. Then we made some specific observations, which we used to describe the KMP string matching algorithm which runs in total $\Theta(n + m)$ time.

Problems and Solution Sketches

8. [CLRS] Give a linear-time algorithm to determine whether a text T is a cyclic rotation of another string T'.

> First, check if $|T| = |T'|$. Then, use the KMP algorithm to check if T occurs in the string $T' \parallel T'$. If so, then T and T' are cyclic rotations of each other. (Prove)